CMSC132: Binary Trees and Binary Search Trees

Chapter 14

By Naren Ayinala

May 2026

Contents

1 Binary Trees and Introduction to BSTs **3**

1.1 What is a Binary Tree? . . . 3

1.2 What Makes a Binary Search Tree Special? . . . 3

1.3 Why Use a BST? . . . 4

2 Some Types of Binary Trees **4**

2.1 Full, Perfect, Complete, and Balanced Trees . . . 4

2.2 Useful Tree Size Formulas . . . 5

3 Tree Traversal **5**

3.1 Breadth First Search (BFS) . . . 6

3.2 Depth First Search (DFS) . . . 6

4 Addition and Removal in a BST **8**

4.1 Searching for a Key . . . 8

4.2 Adding to a BST . . . 8

4.3 Removing from a BST . . . 9

5 Complexity of a BST **11**

6 Complete Sample Code: A Key Value BST **11**

7 Other Useful BST Methods **14**

7.1 Finding the Minimum and Maximum . . . 14

7.2 Counting Nodes and Finding Height . . . 14

8 Practice Questions and Common Mistakes **14**

8.1 Practice: Constructing a BST 14
8.2 Common Mistakes to Avoid 15

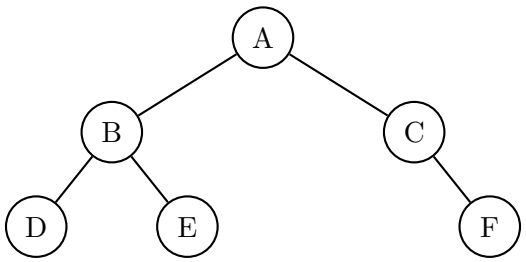
9 Chapter Summary **15**

1 Binary Trees and Introduction to BSTs

1.1 What is a Binary Tree?

As discussed in the introduction to graphs, a **free tree** is a connected graph with no cycles. A **rooted tree** picks one node as the root, giving us parent and child relationships. A **binary tree** is a rooted tree in which every node has **at most two children**. These children are distinguished as the **left child** and the **right child**. A node can have zero, one, or two children. It does not have to have exactly two.

The distinction between left and right matters. Two binary trees with the same values but different left and right placements can be different trees. Unlike a general rooted tree, a binary tree has two specific child positions.



A binary tree. B and C are children of A; C has only a right child.

Vocabulary: The root has no parent. A **leaf** has no children. An **internal node** has at least one child. A **subtree** consists of a node and all of its descendants. A node's **depth** is its number of edges from the root; its **height** is the number of edges on the longest downward path to a leaf. We use the convention that a leaf has height 0 and an empty tree has height -1.

Node	Depth	Height
A	0	2
B, C	1	1
D, E, F	2	0

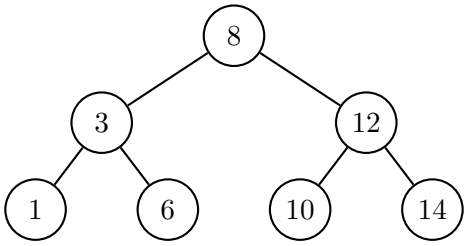
Important: Height conventions can vary between textbooks. Some count nodes rather than edges. Always check the convention used in a problem.

1.2 What Makes a Binary Search Tree Special?

A **Binary Search Tree (BST)** is a binary tree with an additional ordering rule. For *every* node X:

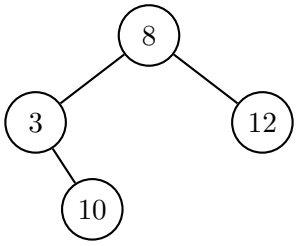
- Every value in X's **entire left subtree** must be smaller than X's value.
- Every value in X's **entire right subtree** must be greater than X's value.
- Both subtrees must themselves satisfy the same BST rule.

For these notes, we use a BST with **unique integer keys**. Real implementations can allow duplicates, but they need a clearly defined policy, such as storing a count or consistently deciding where equal keys go.



Valid BST: every value in the left subtree of 8 is smaller than 8.

Common mistake: It is not enough to check only the immediate children. Imagine a root of 8, its left child 3, and a node 10 somewhere below 3. Even if 10 is greater than its direct parent, it is still in the left subtree of 8, so the tree is *not* a valid BST.



Invalid BST: 10 is in the left subtree of 8.

1.3 Why Use a BST?

Think of searching for a number. At each node, we compare the target to the current key. If the target is smaller, we can ignore the entire right subtree and go left. If it is larger, we can ignore the entire left subtree and go right. This is similar to binary search on a sorted array, except that the BST's shape controls how many comparisons we need.

A regular binary tree does *not* have this ordering rule, so searching for an arbitrary value may require checking every node.

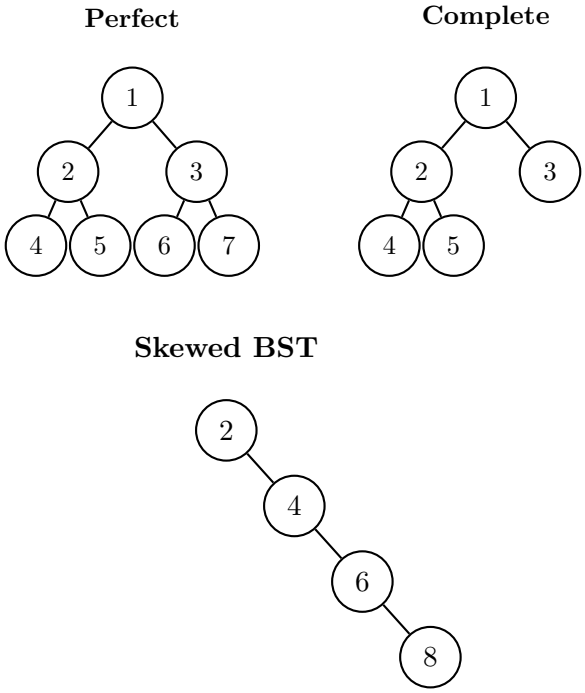
2 Some Types of Binary Trees

2.1 Full, Perfect, Complete, and Balanced Trees

These words sound similar but mean different things.

- **Full binary tree:** Every node has either 0 or 2 children. No node has exactly one child.
- **Perfect binary tree:** Every internal node has 2 children and all leaves are at the same depth. A perfect binary tree is also full and complete.
- **Complete binary tree:** Every level is filled except possibly the last, and the last level fills from left to right with no gaps. This property is especially important for heaps.
- **Balanced binary tree:** The height is controlled so it does not become too large compared to the number of nodes. Different balancing schemes use different exact definitions. AVL and red-black trees are examples of self-balancing BSTs.

- **Degenerate or skewed tree:** Every internal node has only one child. This behaves like a linked list and can make BST operations slow.



2.2 Useful Tree Size Formulas

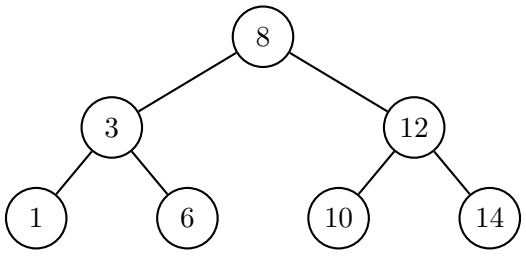
For a perfect binary tree with root at level 0 and height h, the number of nodes on level d is 2^d, and the total number of nodes is

$$n = 1 + 2 + 4 + \cdots + 2^h = 2^{h+1} - 1.$$

For any nonempty binary tree of height h, the number of nodes is at most $2^{h+1} - 1$. A binary tree with n nodes has exactly $n - 1$ edges. These facts help with complexity and counting questions.

3 Tree Traversal

Traversal means visiting all nodes of a tree in a systematic order. We might do this to print the data, find a value, copy the tree, calculate its size, or perform some operation on each node. There are two main strategies: **Breadth First Search (BFS)** and **Depth First Search (DFS)**.



We will use this same tree for every traversal example.

3.1 Breadth First Search (BFS)

BFS goes **level by level**. Basically, visit the root first, then all nodes one level below it, then all nodes at the next level, and so on. For our example the order is

$$\boxed{8,\ 3,\ 12,\ 1,\ 6,\ 10,\ 14}.$$

We normally implement BFS using a **queue**. A queue is first in, first out (FIFO), which means nodes discovered earlier are processed earlier.

1. Add the root to the queue.
2. Remove the front node, visit it, and add its left and right children if they exist.
3. Repeat until the queue is empty.

Step	Node visited	Queue after adding children
0	None	[8]
1	8	[3, 12]
2	3	[12, 1, 6]
3	12	[1, 6, 10, 14]
4	1	[6, 10, 14]
5	6	[10, 14]
6	10	[14]
7	14	[]

```
import java.util.ArrayDeque;
import java.util.Queue;

static void bfs(Node root) {
    if (root == null) return;
    Queue<Node> queue = new ArrayDeque<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        Node current = queue.remove();
        System.out.print(current.key + " ");
        if (current.left != null) queue.add(current.left);
        if (current.right != null) queue.add(current.right);
    }
}
```

The code above assumes the `Node` class defined later in the complete BST implementation. For a tree with n nodes, BFS visits each node once, so the time is $O(n)$. The queue uses $O(w)$ extra space where w is the maximum number of nodes on a level; in the worst case, $w = O(n)$.

3.2 Depth First Search (DFS)

DFS goes down a branch as deeply as possible before backtracking. A **stack** is the natural data structure for DFS, and recursion uses the Java call stack automatically. For binary trees, the three classic DFS orders differ by *when* we process the parent relative to the left and right subtrees.

3.2.1 Preorder Traversal

Order: Parent, Left, Right. Visit the parent first, then recursively visit the left subtree, then the right subtree.

8, 3, 1, 6, 12, 10, 14

```
static void preorder(Node node) {
    if (node == null) return;
    System.out.print(node.key + " ");
    preorder(node.left);
    preorder(node.right);
}
```

Preorder is useful when we want to process the root before its descendants, such as when creating a preorder serialization of a tree (with null markers to preserve shape).

3.2.2 Inorder Traversal

Order: Left, Parent, Right. Visit the entire left subtree, then the parent, then the right subtree.

1, 3, 6, 8, 10, 12, 14

```
static void inorder(Node node) {
    if (node == null) return;
    inorder(node.left);
    System.out.print(node.key + " ");
    inorder(node.right);
}
```

Very important: Inorder traversal of a valid BST with distinct keys produces the keys in **sorted ascending order**. Inorder traversal of an arbitrary binary tree does not necessarily produce sorted values.

3.2.3 Postorder Traversal

Order: Left, Right, Parent. Process both subtrees before processing the parent.

1, 6, 3, 10, 14, 12, 8

```
static void postorder(Node node) {
    if (node == null) return;
    postorder(node.left);
    postorder(node.right);
    System.out.print(node.key + " ");
}
```

Postorder is useful when a parent should be processed only after its children, such as evaluating expression trees or computing subtree information.

3.2.4 Iterative DFS with a Stack

Recursion is convenient, but we can also explicitly create a stack. The following preorder implementation pushes the **right child first** so that the left child is processed first (the stack is last in, first out).

```
import java.util.ArrayDeque;
import java.util.Deque;

static void preorderIterative(Node root) {
    if (root == null) return;
    Deque<Node> stack = new ArrayDeque<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        Node current = stack.pop();
        System.out.print(current.key + " ");
        if (current.right != null) stack.push(current.right);
        if (current.left != null) stack.push(current.left);
    }
}
```

Complexity: Every traversal takes $O(n)$ time because every node is visited once. Recursive DFS uses $O(h)$ auxiliary stack space where h is the tree height measured in edges, up to a constant adjustment for stack frames. A skewed tree can require $O(n)$ stack space.

4 Addition and Removal in a BST

4.1 Searching for a Key

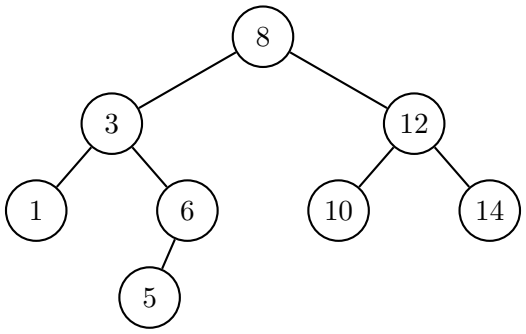
Before discussing insertion, it helps to understand search. Suppose we want to find 10 in the earlier BST. We start at 8. Since $10 > 8$, we move right to 12. Since $10 < 12$, we move left to 10. We found the key after examining three nodes.

```
boolean contains(int key) {
    Node current = root;
    while (current != null) {
        if (key == current.key) return true;
        if (key < current.key) current = current.left;
        else current = current.right;
    }
    return false;
}
```

Notice that we do *not* search both subtrees. The BST ordering tells us which side can possibly contain the key.

4.2 Adding to a BST

When adding a new key, we compare it with the current node and keep moving left or right until we reach an empty child position. Then we create a new node at that position. For example, to insert 5 into the BST, compare 5 with 8 (go left), compare with 3 (go right), compare with 6 (go left), then insert 5.



BST after inserting 5.

```
private Node insert(Node node, int key) {
    if (node == null) return new Node(key);
    if (key < node.key) node.left = insert(node.left, key);
    else if (key > node.key) node.right = insert(node.right, key);
    return node; // duplicate keys are ignored
}

public void add(int key) {
    root = insert(root, key);
}
```

Why return the node? Recursive insertion may create a new subtree root when it reaches `null`. The caller must reconnect that returned node to the parent's `left` or `right` reference. Assigning `root = insert(root, key)` also handles the initially empty tree.

4.3 Removing from a BST

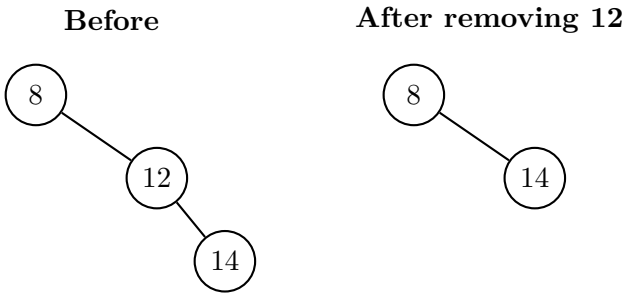
Removal is more complicated because we cannot simply delete a node and leave its children disconnected. We must preserve the BST ordering. There are **three cases**.

4.3.1 Case 1: Node Has No Children

If the node is a leaf, simply remove it by setting its parent's corresponding child reference to `null`. For example, deleting 1 from the original tree just removes that leaf.

4.3.2 Case 2: Node Has Exactly One Child

If the node has one child, connect its parent directly to that child. For example, if 12 has only a right child 14, removing 12 makes 14 take its place. This works because the child's entire subtree already satisfies the ordering constraints for that position.



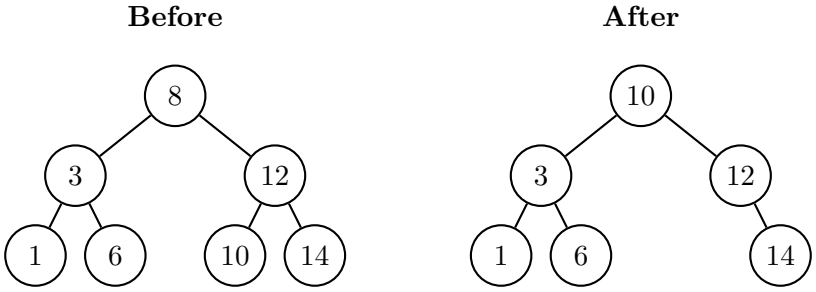
4.3.3 Case 3: Node Has Two Children

If a node has two children, we cannot replace it with either child arbitrarily because that may break the BST property. Instead, we usually choose one of two replacements:

- **Inorder predecessor:** The largest value in the node's left subtree (keep following right pointers).
- **Inorder successor:** The smallest value in the node's right subtree (keep following left pointers).

Both choices preserve the sorted order. After copying the predecessor or successor value into the node, we must remove the original predecessor or successor node, otherwise we would have a duplicate.

Example: Remove 8 from the BST. The inorder successor is 10, which is the smallest value in 8's right subtree. Copy 10 into the root, then remove the original 10 from the right subtree.




```
private Node delete(Node node, int key) {
    if (node == null) return null;
    if (key < node.key) {
        node.left = delete(node.left, key);
    } else if (key > node.key) {
        node.right = delete(node.right, key);
    } else {
        // Zero or one child
        if (node.left == null) return node.right;
        if (node.right == null) return node.left;

        // Two children: replace with inorder successor
        Node successor = node.right;
        while (successor.left != null) {
            successor = successor.left;
        }
        node.key = successor.key;
        node.right = delete(node.right, successor.key);
    }
    return node;
}

public void remove(int key) {
    root = delete(root, key);
}
```

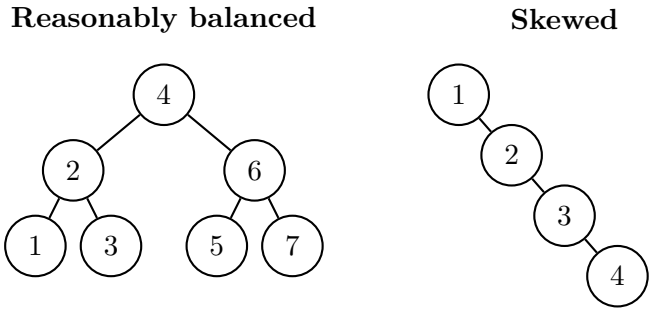
Important: Removing the root uses the same logic as removing any other node. Reassigning `root` to the returned subtree is necessary, especially if the root has zero or one child.

5 Complexity of a BST

The running time for searching, inserting, and removing a key depends on the **height** h of the BST. At each comparison we follow one branch, so the number of steps is proportional to $h+1$. We therefore say these operations take $O(h)$ time.

Operation	Balanced BST	Worst case skewed BST
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Remove	$O(\log n)$	$O(n)$
Inorder traversal	$O(n)$	$O(n)$
Minimum / maximum	$O(\log n)$	$O(n)$

A **balanced** tree has height $O(\log n)$, because each level can hold roughly twice as many nodes as the previous level. But if we insert already sorted values into an ordinary unbalanced BST, the tree may become a chain with height $n-1$.



Correction to a common comparison: A balanced BST does not automatically beat an array at every operation. A sorted array also supports $O(\log n)$ binary search and has $O(1)$ indexed access, while inserting in the middle of an array often costs $O(n)$ because elements must shift. A BST supports efficient ordered insertion and deletion *when its height stays small*. An ordinary BST does not rebalance itself automatically.

6 Complete Sample Code: A Key Value BST

In your original notes, the sample BST is a **key value pair** BST. This means we compare nodes using their keys, but each node can also store associated data. For example, a student ID could be the key and a student name could be the value. If we only wanted to store integer keys, we could remove the `value` field and its related methods.

The following is a complete Java class. It supports insertion, updating an existing key's value, searching, removal, printing in sorted key order, and BFS. The tree is not self balancing.

```
import java.util.ArrayDeque;
import java.util.Queue;

public class BinarySearchTree {
    private static class Node {
        int key;
```

```
        String value;
        Node left, right;

        Node(int key, String value) {
            this.key = key;
            this.value = value;
        }
    }

    private Node root;

    public void put(int key, String value) {
        root = put(root, key, value);
    }

    private Node put(Node node, int key, String value) {
        if (node == null) return new Node(key, value);
        if (key < node.key) {
            node.left = put(node.left, key, value);
        } else if (key > node.key) {
            node.right = put(node.right, key, value);
        } else {
            node.value = value; // update existing key
        }
        return node;
    }

    public String get(int key) {
        Node current = root;
        while (current != null) {
            if (key == current.key) return current.value;
            current = key < current.key
                    ? current.left : current.right;
        }
        return null;
    }

    public boolean containsKey(int key) {
        Node current = root;
        while (current != null) {
            if (key == current.key) return true;
            current = key < current.key
                    ? current.left : current.right;
        }
        return false;
    }

    public void remove(int key) {
        root = remove(root, key);
    }

    private Node remove(Node node, int key) {
        if (node == null) return null;
        if (key < node.key) {
            node.left = remove(node.left, key);
        } else if (key > node.key) {
            node.right = remove(node.right, key);
        } else {
            if (node.left == null) return node.right;
            if (node.right == null) return node.left;
```

```
            Node successor = node.right;
            while (successor.left != null) {
                successor = successor.left;
            }
            node.key = successor.key;
            node.value = successor.value;
            node.right = remove(node.right, successor.key);
        }
        return node;
    }

    public void printInorder() {
        printInorder(root);
        System.out.println();
    }

    private void printInorder(Node node) {
        if (node == null) return;
        printInorder(node.left);
        System.out.print(node.key + ":" + node.value + " ");
        printInorder(node.right);
    }

    public void printBfs() {
        if (root == null) return;
        Queue<Node> queue = new ArrayDeque<>();
        queue.add(root);
        while (!queue.isEmpty()) {
            Node node = queue.remove();
            System.out.print(node.key + " ");
            if (node.left != null) queue.add(node.left);
            if (node.right != null) queue.add(node.right);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        BinarySearchTree bst = new BinarySearchTree();
        bst.put(8, "eight");
        bst.put(3, "three");
        bst.put(12, "twelve");
        bst.put(1, "one");
        bst.put(6, "six");
        bst.put(10, "ten");
        bst.put(14, "fourteen");

        System.out.println(bst.get(6)); // six
        System.out.println(bst.containsKey(5)); // false
        bst.printInorder();
        // 1:one 3:three 6:six 8:eight
        // 10:ten 12:twelve 14:fourteen
        bst.printBfs(); // 8 3 12 1 6 10 14
        bst.remove(8);
        bst.printInorder();
        // 1:one 3:three 6:six 10:ten
        // 12:twelve 14:fourteen
    }
}
```

Note: Returning `null` from `get` means either the key is absent or it is present with a `null` value. Use

`containsKey` to distinguish those cases. The code intentionally stores `String` values for clarity; a generic `BST<K,V>` would be a natural extension after studying generics.

7 Other Useful BST Methods

7.1 Finding the Minimum and Maximum

The smallest key is the leftmost node; the largest is the rightmost node. This works because of the BST ordering.

```
static Node minimum(Node root) {
    if (root == null) return null;
    Node current = root;
    while (current.left != null) current = current.left;
    return current;
}

static Node maximum(Node root) {
    if (root == null) return null;
    Node current = root;
    while (current.right != null) current = current.right;
    return current;
}
```

7.2 Counting Nodes and Finding Height

Both are natural examples of recursive tree problems. First solve the problem for the left subtree, then the right subtree, and combine the answers.

```
static int size(Node node) {
    if (node == null) return 0;
    return 1 + size(node.left) + size(node.right);
}

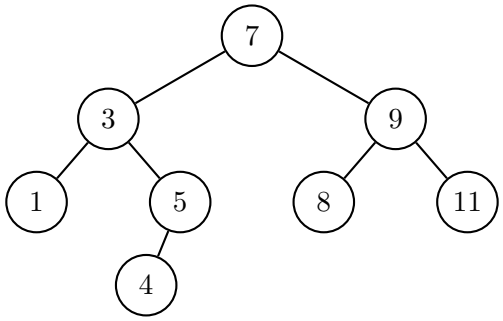
static int height(Node node) {
    if (node == null) return -1; // height in edges
    return 1 + Math.max(height(node.left),
                        height(node.right));
}
```

Complexity: Both methods visit all nodes, so they take $O(n)$ time. Their recursive stack usage is $O(h)$.

8 Practice Questions and Common Mistakes

8.1 Practice: Constructing a BST

Insert the keys $7, 3, 9, 1, 5, 8, 11, 4$ in that order into an initially empty BST. Then write its preorder, inorder, postorder, and BFS traversals.



Answers:

- Preorder: $7, 3, 1, 5, 4, 9, 8, 11$.
- Inorder: $1, 3, 4, 5, 7, 8, 9, 11$.
- Postorder: $1, 4, 5, 3, 8, 11, 9, 7$.
- BFS: $7, 3, 9, 1, 5, 8, 11, 4$.

8.2 Common Mistakes to Avoid

1. **Checking only immediate children:** The BST condition applies to every value in the entire left and right subtrees.
2. **Thinking every binary tree is a BST:** A binary tree only limits the number of children. A BST additionally enforces ordering.
3. **Confusing BFS and DFS:** BFS uses a queue to explore levels; DFS uses recursion or a stack to explore branches.
4. **Forgetting to reconnect references:** In recursive insertion and removal, always assign the returned subtree root to the appropriate child reference.
5. **Forgetting the second deletion step:** After copying the inorder successor or predecessor into a node with two children, remove the original replacement node.
6. **Assuming all BST operations are $O(\log n)$:** They are $O(h)$, and an unbalanced tree can have $h = O(n)$.
7. **Assuming inorder is always sorted:** It is sorted for a valid BST under the chosen ordering policy, not for an arbitrary binary tree.

9 Chapter Summary

A binary tree is a rooted tree with at most two ordered child positions per node. A BST is a binary tree where every left subtree contains smaller keys and every right subtree contains larger keys. Traversals let us visit all nodes using either BFS (level by level) or DFS (preorder, inorder, or postorder). Search and insertion follow a single BST branch. Deletion has three cases: leaf, one child, and two children. Finally, BST efficiency depends on tree height: balanced trees support $O(\log n)$ operations, but a skewed BST can take $O(n)$.